Actividad autónoma 09

Diferencias por plataforma en PharmaMobil

Grease Duran Castro

Desarrollo de Aplicaciones Móviles - Ciclo VI

Sesión 09 | 11 de octubre de 2026

Proyecto: PharmaMobil sobre PharmaSoft

https://github.com/GreysiLL/SE3AA/tree/feature/autonoma09-duran

Alcance de la evidencia

Android: APK compilado, 69 pruebas aprobadas y ejecución real en Pixel 9 API36. Las tres capacidades tienen capturas propias.

iOS: código implementado, pendiente de compilar y ejecutar por falta de Mac/Xcode. No hay capturas ni un resultado literal de moneda en iOS. Esta limitación afecta los criterios de ejecución en ambas plataformas de la guía.

No se ha coordinado una ejecución con otra persona; nombre del ejecutor iOS pendiente. El documento registra lo efectivamente comprobado.

1. Inventario completo de capacidades

Se recorrieron todos los source sets del proyecto. Se encontraron cuatro declaraciones expect propias. Compartidor se incluye como capacidad nativa con interfaz común e inyección, aclarando que no es expect.

Moneda

expect fun formatearSoles(valor: Double): String

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.kt

androidMain: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.android.kt

iosMain: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.ios.kt

Android: NumberFormat + Locale(es, PE). iOS: NSNumberFormatter + NSLocale(es_PE).

Información del dispositivo

expect class InfoDispositivo() { val sistema: String; val version: String }

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.kt

androidMain: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.android.kt

iosMain: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.ios.kt

Android: Build.VERSION.RELEASE. iOS: UIDevice.systemName y systemVersion.

Inyección y red

expect val platformModule: Module

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/di/AppModule.kt

androidMain: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/di/PlatformModule.android.kt

iosMain: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/di/PlatformModule.ios.kt

Android: Koin androidContext, OkHttp, 10.0.2.2. iOS: Koin sin Context, Darwin, localhost.

El módulo registra el motor de Ktor, la URL del backend y Compartidor. Las APIs HTTP y reglas de negocio permanecen comunes.

Atrás en detalle

@Composable expect fun AtrasDetalle(habilitado: Boolean, volver: () -> Unit)

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/AtrasDetalle.kt

androidMain: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/AtrasDetalle.android.kt

iosMain: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/AtrasDetalle.ios.kt

Android: BackHandler de Activity Compose. iOS: Sin interceptor: botón Volver común.

Compartir (interfaz con DI)

interface Compartidor { fun compartir(texto: String) }

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/domain/platform/Compartidor.kt

androidMain: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/CompartidorAndroid.kt

iosMain: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobil/platform/CompartidorIos.kt

Android: Intent.ACTION_SEND, Context, EXTRA_TEXT. iOS: UIActivityViewController, UIWindowScene.

2. Informe comparativo | 677 palabras

1. Formato de moneda y resultado observado

En PharmaMobil, commonMain declara formatearSoles(valor: Double): String en platform/Formato.kt. Android resuelve esa firma mediante NumberFormat.getCurrencyInstance(Locale("es", "PE")); iOS utiliza NSNumberFormatter, NSNumberFormatterCurrencyStyle y NSLocale("es_PE"). El locale identifica la convención solicitada, pero no obliga a dos bibliotecas distintas a devolver una secuencia idéntica de caracteres. Android utiliza su implementación de internacionalización del entorno Java; iOS utiliza Foundation. También influyen las versiones del sistema y sus datos regionales. Por eso la comparación debe observar símbolos, separadores y espacios, además del valor numérico.

El emulador Android 16 mostró literalmente «S/ 4.50» para Paracetamol 500 mg. La extracción de la interfaz permite identificar el carácter entre la barra y el número: es U+00A0, un espacio de no separación, aunque visualmente parece un espacio ordinario. Este resultado proviene de la función real usada por el detalle, no de un texto escrito manualmente. No se obtuvo un resultado literal en iOS porque no hay acceso a macOS y Xcode. No sería correcto afirmar que Foundation produjo «S/4.50» sin ejecutarlo. El código iOS está preparado y el informe deja esa observación pendiente.

2. Context en Android y presentación en UIKit

CompartidorAndroid recibe un Context para iniciar un Intent.ACTION_SEND, con MIME text/plain y EXTRA_TEXT. platformModule.android.kt entrega androidContext() mediante Koin. Como ese contexto pertenece a la aplicación, la apertura del selector incorpora FLAG_ACTIVITY_NEW_TASK. Esta dependencia refleja el modelo de componentes de Android: iniciar una actividad necesita un contexto desde el que solicitar al sistema la navegación. El detalle común solamente llama compartir(texto); no maneja actividades ni conoce esas constantes.

CompartidorIos no recibe un Context porque UIKit presenta controladores desde otro controlador. Su implementación busca una UIWindowScene activa, su ventana principal y el controlador que puede presentar la hoja. Construye UIActivityViewController y configura el popover para iPad antes de presentarlo. Koin registra CompartidorIos() sin contexto Android. Eso no significa que iOS carezca de dependencias de entorno: requiere una ventana y un presentador activos. La diferencia está en cómo obtiene el entorno de presentación. La captura Android demuestra el texto y el selector; el comportamiento UIKit aún requiere ejecución real.

3. expect/actual frente a una interfaz inyectada

El propio proyecto combina ambas estrategias. Compartidor es una interfaz ordinaria de dominio, no una declaración expect; sus implementaciones se seleccionan en platformModule, que sí es expect/actual. Esto permite probar DetalleProductoViewModel con un compartidor falso y comprobar el texto sin abrir un selector. El acoplamiento del caso de uso queda limitado al contrato de compartir, mientras las clases nativas permanecen en sus source sets.

Para formatearSoles se eligió una función expect porque la operación es pequeña y su llamada resulta directa y legible. Una interfaz FormateadorMoneda con inyección permitiría sustituir el formato en pruebas y agregar políticas distintas, pero exigiría una instancia y registros adicionales. InfoDispositivo sigue el patrón expect class indicado en la guía, con propiedades String que Compose puede mostrar sin tipos nativos. Esa elección simplifica una lectura local; si luego necesitáramos simular versiones o refrescar datos complejos, una interfaz inyectada podría facilitar las pruebas. Ninguna estrategia elimina la necesidad de aislar las APIs específicas.

4. Error real al retirar un actual

Se comentó temporalmente el contenido de Formato.android.kt y se ejecutó :shared:compileAndroidMain en la rama de esta actividad. El compilador informó exactamente: «Expected formatearSoles has no actual declaration in module <commonMain> for JVM». El registro completo conserva la ruta Formato.kt:3:1 y BUILD FAILED. El contrato común existía, pero Android no tenía una implementación que pudiera enlazar. Se restauró el archivo inmediatamente y se volvió a compilar: BUILD SUCCESSFUL. El APK usado para las capturas contiene el actual completo; el cambio deliberadamente incorrecto no quedó en el repositorio.

5. Qué debe permanecer común

La validación de productos y la traducción de errores REST no deberían duplicarse por plataforma. Validar nombre, precio y stock responde a reglas de PharmaMobil, no a Build.VERSION ni a UIKit. Lo mismo ocurre con ErrorApi, los DTO y las transiciones del ViewModel: Android e iOS deben interpretar las mismas respuestas de PharmaSoft y conservar el mismo comportamiento del formulario. Duplicar estas reglas permitiría que una plataforma aceptara un producto que la otra rechaza. El criterio aplicado es mantener común la decisión de negocio y mover solamente el acceso a capacidades del sistema. La búsqueda de importaciones android. y platform. en commonMain dio cero coincidencias.

3. Tercera capacidad: información del dispositivo

Opción elegida de la guía: mostrar sistema operativo y versión en Acerca de. La navegación está conectada desde el menú lateral, conservando los destinos anteriores. No necesita permisos adicionales ni acceso a identificadores personales.

commonMain

```
package pe.edu.upeu.pharmamobil.platform

expect class InfoDispositivo() {
val sistema: String
val version: String
}
```

androidMain

```
package pe.edu.upeu.pharmamobil.platform

import android.os.Build

actual class InfoDispositivo actual constructor() {
actual val sistema: String = "Android"
actual val version: String get() = Build.VERSION.RELEASE
}
```

iosMain

```
package pe.edu.upeu.pharmamobil.platform

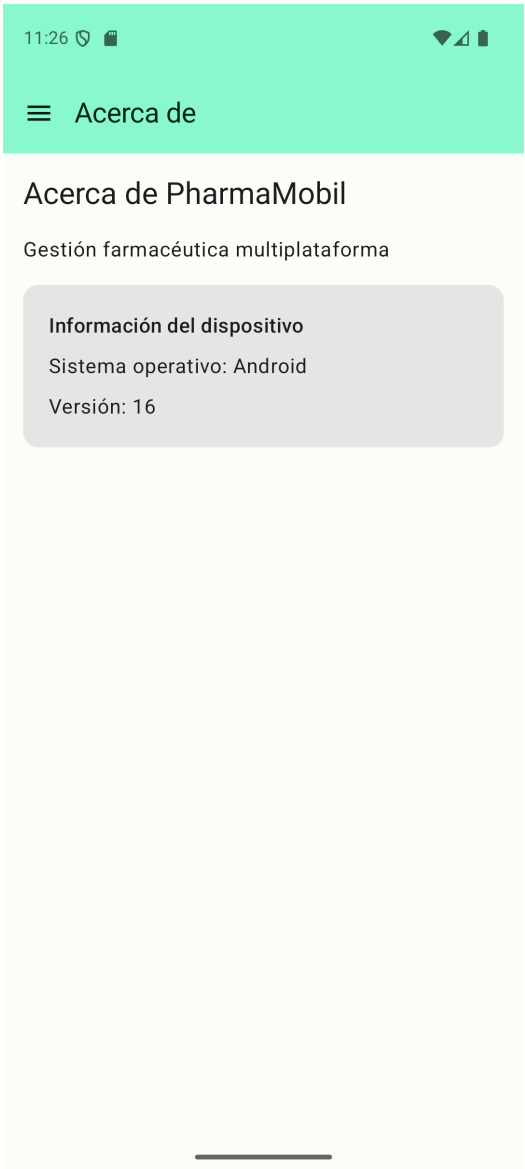
import platform.UIKit.UIDevice

actual class InfoDispositivo actual constructor() {
actual val sistema: String get() = UIDevice.currentDevice.systemName
actual val version: String get() = UIDevice.currentDevice.systemVersion
}
```

AcercaScreen usa remember { InfoDispositivo() } y muestra sus propiedades en una tarjeta común. Android obtiene la versión de Build.VERSION.RELEASE; iOS consulta UIDevice.currentDevice. Ambas firmas conservan el mismo paquete y constructor.

Aislamiento comprobado: cero importaciones android. o platform. en commonMain. La implementación iOS requiere todavía validar su compilación en una Mac.

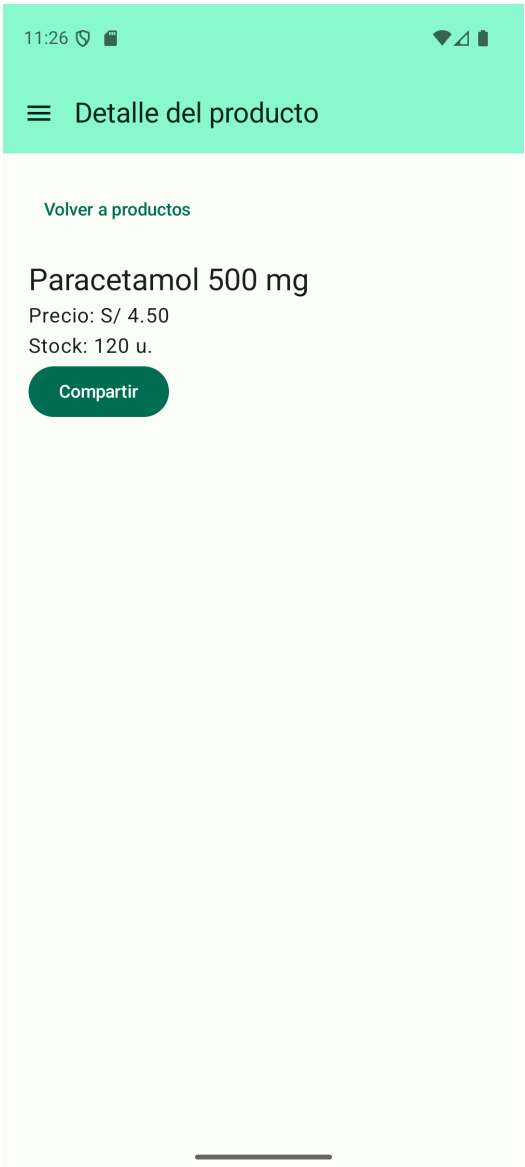
4. Evidencia: información del dispositivo



Android 16. Captura del APK nuevo: menú Acerca de, sistema operativo Android y versión 16. Fecha y textos literales en bitacora.json.

iOS: sin evidencia de ejecución. Pendiente abrir Acerca de en un simulador con Xcode y registrar sistema y versión reales.

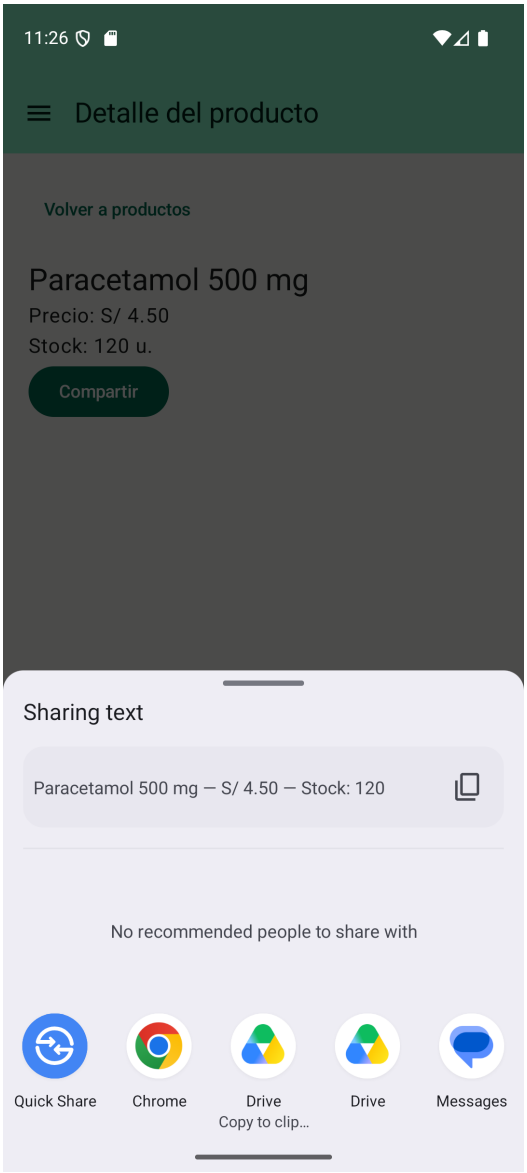
5. Evidencia: moneda del mismo producto



Producto 1: Paracetamol 500 mg. Precio literal Android: S/ 4.50; stock 120. El espacio después de S/ es U+00A0. La captura proviene del detalle conectado por GET a PharmaSoft.

Comparación iOS pendiente: falta ejecutar el mismo producto y registrar la cadena devuelta por NSNumberFormatter, incluidos los caracteres de espacio. No se reproduce un valor supuesto.

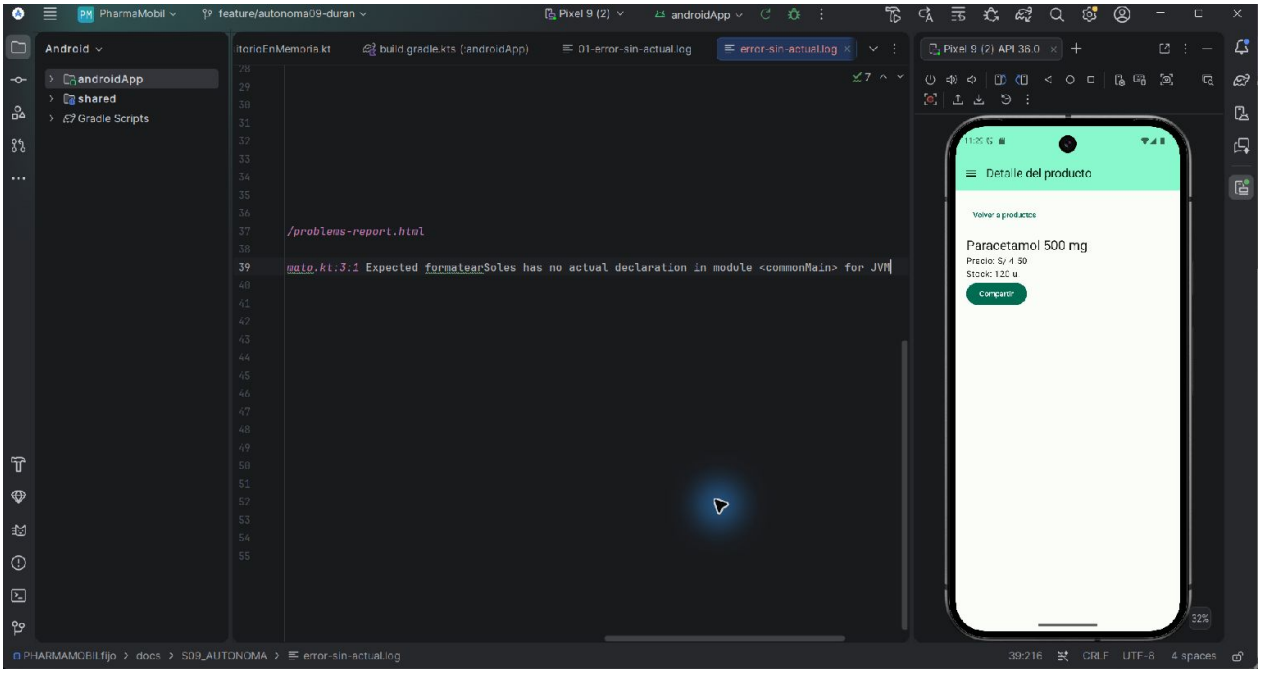
6. Evidencia: selector de compartir



El selector Android muestra: Paracetamol 500 mg — S/ 4.50 — Stock: 120. Se abrió el selector y se canceló sin enviar el producto a ninguna aplicación.

iOS: UIActivityViewController está implementado; no hay captura de su hoja por falta de Mac. Se debe verificar el texto del mismo producto y la presentación en simulador.

7. Evidencia: fallo deliberado y restauración



Mensaje literal: Expected formatearSoles has no actual declaration in module <commonMain> for JVM

Ubicación: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.kt:3:1. Comando: gradlew.bat :shared:compileAndroidMain, con el actual Android comentado temporalmente.

Resultado: BUILD FAILED in 54s. Se restauró exactamente el contenido original en un bloque finally y la compilación posterior terminó BUILD SUCCESSFUL in 31s. La captura muestra el registro nuevo de S09_AUTONOMA abierto en Android Studio.

Registros completos: error-sin-actual.log y compilacion-restaurada.log. El código entregado conserva ambos actual de moneda.

8. Verificación, repositorio y commits

APK Android: BUILD SUCCESSFUL in 1m 24s. 69 pruebas, 0 fallos, 0 errores y 0 omitidas. Resultados JUnit reales resumidos en pruebas.json; log completo en pruebas-y-compilacion.log. La nueva pantalla se verificó en el emulador instalado con este APK.

https://github.com/GreysiLL/SE3AA/tree/feature/autonoma09-duran

feat(aa09): muestra informacion nativa del dispositivo en Acerca de

17cdaf376a3c519758b998a501c6706d36e4d462

docs(aa09): inventaria capacidades y aislamiento por plataforma

f792bcfca5d8313b6a606b77760d121edf764d47

docs(aa09): compara APIs nativas y contrato expect actual

ab7ee5c2b38b2f07ca6f6db27bce3ddc5a001e54

Un cuarto commit registra este documento y las evidencias. Los enlaces se habilitan tras subir la rama; la subida debe comprobarse contra la referencia remota.

Lista de cotejo

Portada e inventario con firmas, rutas, source sets y APIs: preparado.

Informe de 677 palabras con cinco respuestas y error literal: preparado.

Tercera capacidad: implementada para Android e iOS; ejecutada en Android.

Código nativo aislado y README actualizado: comprobado.

Capturas Android: moneda, compartir, dispositivo y compilador: incluidas.

Capturas iOS y comparación literal Foundation: pendientes.

9. Referencias y continuidad de iOS

Kotlin: declaraciones expect/actual

https://kotlinlang.org/docs/multiplatform/multiplatform-expect-actual.html

Android: Build.VERSION.RELEASE

https://developer.android.com/reference/android/os/Build.VERSION#RELEASE

Apple: UIDevice.systemVersion

https://developer.apple.com/documentation/uikit/uidevice/systemversion

Apple: NSNumberFormatter

https://developer.apple.com/documentation/foundation/numberformatter

Fuente de requisitos: guía de la Actividad Autónoma 09 entregada por el usuario, ocho páginas. Las observaciones Android se respaldan en capturas y registros propios; la documentación oficial explica los contratos, pero no sustituye una ejecución iOS.

Pasos pendientes en una Mac

Abrir iosApp con Xcode y ejecutar en simulador. Configurar un PharmaSoft accesible desde esa Mac. Abrir el mismo producto 1, capturar el precio y extraer sus caracteres. Abrir la hoja de compartir y mostrar el texto. Abrir Acerca de y capturar sistema y versión. Registrar el nombre de quien ejecutó, fecha, versión del simulador y resultados antes de declarar verificación en ambas plataformas.